

تقييم تقني شامل

نظام رؤى للتداول الآلي

من التحليل إلى التنفيذ: تقييم معمق للبنية والمنطق والمخاطر

يونيو 2026 | الإصدار 1.0

تقييم شامل لنظام التداول الآلي "رؤى" يغطي +30 وحدة برمجية، 7 استراتيجيات، نظامين تداول مستقلين، و3 طبقات إدارة مخاطر. يتضمن تحليل الثغرات الحرجة ومنها إغلاق الصفقات عند 4 ساعات بدلاً من 48 ساعة، وتوصيات مرتبطة بالأولوية للتحسين.

1 الملخص التنفيذي

5

ثغرات حرجية

+30

وحدة برمجية

7.2

التقييم العام /10

نظام رؤى (Roua Trading) هو منصة تداول آلي متقدمة مبنية على معمارية Modular Monolith باستخدام NestJS و Next.js و Prisma. يتميز النظام بتصميم دفاعي متعدد الطبقات وإدارة مخاطر صارمة، لكنه يعاني من عدة مشاكل حرجية أبرزها إغلاق الصفقات تلقائياً عند 4 ساعات بدلاً من 48 ساعة، وتناقضات في حساب قيمة المحفظة عبر طبقات المخاطر المختلفة، وغياب معالجة التنفيذ الجزئي للأوامر.

نقاط القوة الرئيسية Key Strengths

+ معمارية واضحة الفصل بين نظامي التداول (Agent و Smart Executor) مع تنسيق عبر Redis

+ 3 طبقات إدارة مخاطر مستقلة مع قواعد غير قابلة للتجاوز (SL إجباري، حد خسارة يومي)

+ نظام مصالحة المراكز (Reconciliation) لمنع فقدان الصفقات المنفذة على البورصة

+ قفل متفائل (Optimistic Locking) لمنع الإغلاق المتزامن المزدوج للمراكز

+ توثيق مكثف داخل الكود مع تتبع الإصدارات (V133، V176، V214...)

+ دعم +30 لغة في الواجهة و PWA للجوال

المخاوف الحرجة Critical Concerns

! إغلاق جميع الصفقات (Agent + Smart) عند 4 ساعات مع `closeReason="Manual"` بدلاً من `TIME_EXPIRED`

! كود قديم يعمل على Railway رغم الإصلاحات المحلية - مشكلة نشر خطيرة

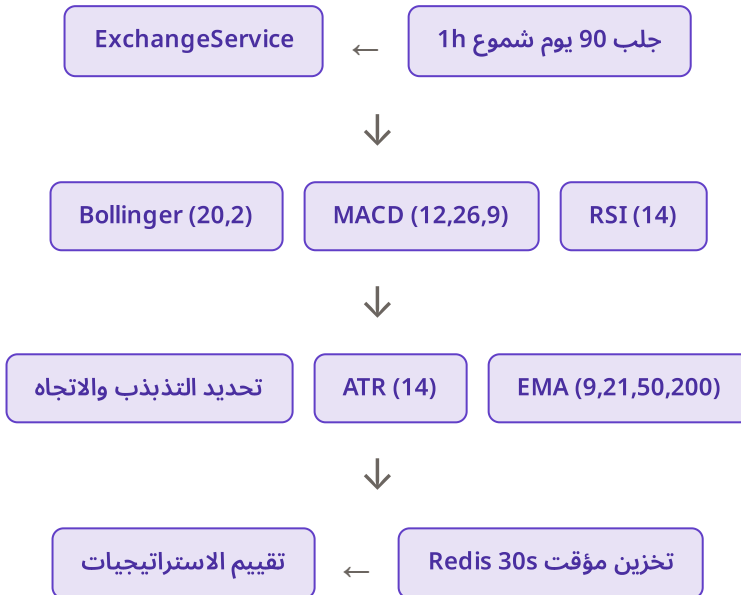
! تناقض في حساب قيمة المحفظة بين RiskCalculator و RiskManager يؤثر على قرارات التداول

! Exchange Sync يغلق المراكز بدون `closeReason` مما يسجلها كـ `AUTO_CLOSE` بشكل افتراضي

! غياب معالجة التنفيذ الجزئي (Partial Fills) رغم وجود الحالة في ال Enum

2 تقييم طبقة التحليل والاستراتيجيات

خط أنابيب تحليل السوق Market Analysis Pipeline



يتميز خط أنابيب التحليل بتغطية شاملة للمؤشرات الفنية مع تخزين مؤقت ذكي عبر Redis لمدة 30 ثانية، مما يقلل من استدعاءات API المتكررة. المؤشرات تفوض إلى تطبيقات معيارية في `indicator-` `algorithms.utils.ts` مما يضمن تناسقاً في الحسابات عبر جميع الخدمات.

تحذير: البديل التحليلي المحدود

الدالة `_buildMinimalAnalysis()` تولّد مؤشرات اصطناعية من نسبة التغير اليومي فقط عند توفر أقل من 50 شمعة. هذا يمكن أن ينتج إشارات مضللة، حيث تُضخّم قيم RSI و Bollinger Bands بشكل مصطنع لتمرير الإشارات عبر الاستراتيجيات. يُنصح بمنع توليد الإشارات تماماً عندما تكون البيانات غير كافية بدلاً من استخدام بيانات اصطناعية.

نظام الاستراتيجيات السبع Seven Strategy System

الاستراتيجية	نسبة المخاطرة/العائد	الإطار الزمني	الاستخدام	التقييم
SWING	1.5	H1-H4	Agent فقط	ممتاز
MOMENTUM_BREAKOUT	1.2	M30-H4	Agent فقط	جيد
VWAP_RSI	1.0	M15-H1	كلاهما	جيد
SCALPING	1.0	M1-M15	Smart فقط	مقبول
MEAN_REVERSION	0.8	M15-H1	Agent فقط	حذر
GRID	0.8	M5-M30	Agent فقط	حذر

الاستراتيجية	نسبة المخاطرة/العائد	الإطار الزمني	الاستخدام	التقييم
DCA	0.4	M15-H4	Agent فقط	ضعيف

نظام الاستراتيجيات يُظهر تصميمًا ناضجاً مع فصل واضح بين استراتيجيات (M30-W1) Agent و Smart Executor (M1-M15). وضع AUTO يكشف نظام السوق ويقيم جميع الاستراتيجيات ثم يختار الأفضل. آلية الهوية (Idempotency) بنافذة 30 ثانية تمنع الإشارات المكررة بشكل فعال.

مشكلة: استراتيجية DCA بنسبة مخاطرة/عائد 0.4

استراتيجية DCA تحمل أقل نسبة مخاطرة/عائد (0.4) مما يعني أن العائد المتوقع أقل من المخاطرة المتخذة. هذا غير منطقي لاستراتيجية متوسطة المدى وقد يؤدي إلى تراكم الخسائر. يُنصح برفع الحد الأدنى إلى 0.8 على الأقل أو إيقاف الاستراتيجية حتى تحسينها.

كشف نظام السوق

Regime Detection

نظام Adaptive Strategy Selector يكشف الحالة الحالية للسوق (اتجاه صاعد/هابط/عرضي، تذبذب منخفض/متوسط/مرتفع/حاد) ويختار الاستراتيجية الأنسب تلقائياً. هذا يعزز من مرونة النظام بشكل كبير، لكن نقطة الضعف هي أن بيانات التدريب تعتمد كلياً على الأداء التاريخي داخل النظام نفسه دون backtesting خارجي، مما قد يخلق تحيزاً دائرياً في اختيار الاستراتيجيات.

3

تقييم طبقة إدارة المخاطر

البنية ثلاثية الطبقات

Three-Layer Architecture

الطبقة	الخدمة	النطاق	التقييم
قبل التداول (Agent)	RiskCalculatorService	تقييم لكل إشارة، يحسب فقط صفقات Agent	جيد
قبل التداول (عام)	RiskGatekeeperService	يتحقق من كل طلب، circuit breakers، فصل حسب المصدر	جيد
أثناء التنفيذ	RiskManagerService	فحص أخير قبل البورصة، معلمات مخاطر من DB	تناقضات

القواعد غير القابلة للتجاوز

Non-Negotiable Rules

القواعد المُنفَّذة بشكل صحيح	التناقضات والمشاكل
- وقف خسارة إجباري على 4 مستويات - حد حجم المركز: 2% من المحفظة - حد خسارة يومية: 5% مع إيقاف تلقائي - حد الصفقات المفتوحة: 5 (Agent) / 20 (إجمالي) - Circuit breaker: 3 خسائر متتالية = توقف 30 دقيقة - منع التكرار: 5 دقائق بين صفقات نفس الرمز - مفتاح إيقاف شامل: AUTO_TRADING_ENABLED	- حساب قيمة المحفظة مختلف بين RiskCalculator و RiskManager - فحص مخاطر مزدوج وربما متعارض - Auto-migration صامتة لإعدادات المخاطر في RiskManager - معامل مخاطرة عشوائي (3 × riskPerTrade) - Fail-open في TradeCoordinationService عند تعطل Redis - عدم تناسق في سقف الصفقات بين OrderDispatcher و ExposureManager

ثغرة حرجية: تناقض حساب قيمة المحفظة

`RiskCalculatorService._getPortfolioValue()` يضيف الأرباح/الخسائر غير المحققة إلى `paperBalance`، بينما `RiskManagerService._estimatePortfolioValue()` يضيف القيمة الإسمية الكاملة للمركز. هذا ينتج قيماً مختلفة لنفس المستخدم، مما يعني أن قرار فتح مركز قد يُرفض من طبقة ويُقبل من أخرى. الحل: توحيد طريقة الحساب في خدمة واحدة مشتركة.

تحذير: bug V204 في RiskManager

`_estimatePortfolioValue()` يرجع `paperBalance.toNumber()` الذي قد يكون 0 إذا لم يعيّن المستخدم رصيداً ورقياً، مما يمنع جميع التداول. القيمة الافتراضية 10000 تُطبّق فقط في `catch block`، وليس كقيمة افتراضية صريحة.

Circuit Breakers

نظام Circuit Breaker يعمل بمستوى المستخدم والرمز معاً (V137 fix): تغيير من مفاتيح عامة إلى مفاتيح خاصة بالمستخدم لمنع التلوث بين المستخدمين). عند 3 خسائر متتالية على نفس الرمز خلال 5 دقائق، يُفعّل نظام التبريد لمدة 30 دقيقة. هذا تصميم سليم يحمي من الانهيارات المتسلسلة، لكن عتبة 3 خسائر قد تكون منخفضة جداً لتداول Scalping حيث التقلبات الطبيعية قد تنتج 3 خسائر متتالية.

4 تقييم طبقة التنفيذ وإدارة المراكز

مسار التنفيذ Execution Paths

Autonomous Trader Agent

- إطارات: M30, H1, H4, D1, W1
- مصدر الإشارة: تحليل ذاتي + تقييم استراتيجيات
- تكرار الفحص: كل 60 ثانية
- استراتيجيات: 6 (..., AUTO/SWING/GRID)
- أقصى مدة: 48 ساعة (V214)

Smart Executor (Scalping)

- إطارات: M1, M5, M15
- مصدر الإشارة: Strategic Council Briefs
- تكرار الفحص: كل 10 ثوان
- استراتيجية: Scalping فقط
- أقصى مدة: 4 ساعات (M1/M5)

مراقبة المراكز Position Monitoring

تعمل خدمة Position Monitor كل 10 ثوان وتُفحص: ضرب SL، ضرب TP، التريلينج ستوب (ينشط عند 2% ربح ويتبع بـ 1%)، وحد أقصى لمدة الاحتفاظ حسب الإطار الزمني. للصفقات الورقية (paper-trading) القديمة أكثر من 48 ساعة بدون SL/TP، يتم إغلاقها تلقائياً كـ STALE_POSITION.

ثغرة حرجية: إغلاق الصفقات عند 4 ساعات بدلاً من 48 ساعة

الظاهرة: جميع صفقات Agent و Smart Executor تُغلق عند 4 ساعات بالضبط مع `closeReason = "Manual"` بدلاً من `"TIME_EXPIRED"`.

التحليل الجذري:

1. الكود المحلي تم إصلاحه بالكامل (V184) أزال إغلاق 4h من V213 Agent. أضاف V214 safety net. أضاف حماية في `closePosition`.

2. لكن الكود القديم (pre-V184) لا يزال يعمل على Railway. الدليل: `closeReason="Manual"` كان القيمة الافتراضية قبل V176، مما يعني أن الكود الفعال على Railway أقدم من V176.

3. `ExchangeSyncService._closePositionInDB()` يستدعي `closePositionWithRetry()` بدون `closeReason`، مما يجعله يُسجل كـ `AUTO_CLOSE` (بعد V176) أو `MANUAL` (قبل V176).

الحل المطلوب: إعادة نشر فورية للإصدار الأحدث على Railway مع التحقق من أن ملفات JS المُجمّعة محدّثة فعلاً.

حد أقصى لمدة الاحتفاظ Max Holding Time

الإطار الزمني	أقصى مدة	التصنيف	الحالة
M1 / M5	4 ساعات	Scalping	صحيح
M15 / M30	12 ساعة	Intraday	صحيح

الإطار الزمني	أقصى مدة	التصنيف	الحالة
H1 / H4	48 ساعة	Swing	صحيح
D1 / W1	7-14 يوم	Position	صحيح
Agent (أي إطار)	48 ساعة	Swing Default	لا يعمل على Railway

القفل المتفائل والمصالحة Optimistic Locking & Reconciliation

يستخدم النظام قفلاً متفائلاً (Optimistic Locking) عبر حقل `version` في جدول Position لمنع الإغلاق المتزامن المزدوج. عند فشل القفل، تتم إعادة المحاولة حتى 3 مرات. خدمة Reconciliation تعمل كل 30 ثانية لإعادة محاولة تحديثات المراكز الفاشلة (حد أقصى 5 محاولات ثم تُعلّم كـ FAILED). هذا تصميم متين يمنع "الصفقات المفقودة" - المراكز المنفذة على البورصة بدون سجل في قاعدة البيانات.

تحذير: غياب معالجة التنفيذ الجزئي

حالة `PARTIALLY_FILLED` موجودة في الـ Enum لكن لا يوجد أي منطق لمعالجتها. النظام يفترض تنفيذاً كاملاً دائماً. في التداول الحقيقي على البورصات ذات السيولة المنخفضة، التنفيذ الجزئي شائع وقد يؤدي إلى مراكز بحجم مختلف عن المتوقع.

5 تقييم النظام المزدوج: Agent vs Smart Executor

فصل المهام Separation of Concerns

يُعدّ الفصل بين نظامي التداول من أبرز نقاط قوة النظام. Smart Executor مخصص لل Scalping السريع (M1-M15) ويتلقى إشارات من Strategic Council، بينما Agent يعمل كمتداول مستقل يحلل السوق بنفسه ويتخذ قراراته على أفق زمني أطول (M30-W1). كل نظام يراقب صفقاته فقط (source='smart_executor') و (source='agent')، مما يمنع تدخل أحدهما في صفقات الآخر.

مجلس الاستراتيجية Strategic Council

يعمل المجلس كـ "القائد في غرفة العمليات" - يحلل السوق بعمق ويصدر Trading Briefs لل Smart Executor. لا ينفذ صفقات مباشرة بل يوجّه فقط. يستخدم إجماع نماذج AI متعددة (حتى 8 نماذج) مع عتبة ثقة 50% وعتبة إجماع 55%. هذا يقلل من تأثير تحيز نموذج واحد ويعزز موثوقية الإشارات.

التنسيق بين النظامين Cross-System Coordination

تناقض في قواعد التنسيق

TradeCoordinationService.canOpenPosition() يمنع مصادر مختلفة من فتح صفقات على نفس الرمز، لكن OrderDispatcherService يسمح بصفقات متقابلة (hedge) من مصادر مختلفة باتجاه معاكس. هاتان القاعدتان تتعارضان وقد تؤدي إحداهما إلى حظر صفقة مسموح بها من الأخرى. كذلك، TradeCoordinationService يستخدم fail-open عند تعطل Redis، مما يعني أن تعطل Redis يسمح بفتح صفقات مكررة من كلا النظامين على نفس الرمز.

جدول مقارنة تفصيلي Detailed Comparison

البعد	Smart Executor	Agent
الأطر الزمنية	M1, M5, M15	M30, H1, H4, D1, W1
مصدر الإشارة	Strategic Council Briefs	تحليل ذاتي + تقييم استراتيجي
تكرار التشغيل	10 ثوان	60 ثانية
الاستراتيجيات	Scalping فقط	7 استراتيجيات + AUTO
تخزين الحالة	Redis per-user	Redis + DB AgentSession
تتبع المصدر	source='smart_executor'	source='agent'
نظام المخاطر	RiskGatekeeper + RiskManager	RiskCalculator + RiskGatekeeper + RiskManager
الذكاء الاصطناعي	يقرأ Council Briefs	تحليل ذاتي + StrategicCouncil اختياري

ملاحظة إيجابية: فصل المراقبة

كل نظام يراقب صفقاته فقط (Agent يراقب PositionMonitor ، source='agent ' يراقب الكل). هذا يمنع المشكلة التي كانت تحدث سابقاً حيث كان Agent يغلق صفقات Smart Executor بالخطأ (V133 fix).

6 تقييم البنية التحتية والهندسة

البنية التقنية Tech Stack

المكون	التقنية	التقييم
الخادم	NestJS 11 + TypeScript	ممتاز
الواجهة	Next.js 16 + React 18 + TailwindCSS 4	ممتاز
قاعدة البيانات	PostgreSQL + Prisma 6.19	ممتاز
التخزين المؤقت	Redis (ioredis) + BullMQ (معطل)	حذر
بيانات السوق	CCXT 4.4 + TwelveData + MetaAPI	جيد
نماذج AI	+10 مزود (OpenRouter, Gemini, DeepSeek, ...)	ممتاز
المصادقة	WebAuthn (Passkeys) + Google OAuth	ممتاز

تصميم قاعدة البيانات Database Design

مخطط Prisma يتضمن +30 نموذجاً مع تصميم محكم: دقة عشرية (Decimal 19,8) للكميات، تتبع المصدر (source) على كل جدول، قفل متفائل (version) على Position، وعزل على مستوى الصفوف (RLS) مع `UserIsolationInterceptor`. جدول `PositionReconciliation` هو نمط إنتاجي ممتاز للتعامل مع الإخفاقات الجزئية في الأنظمة الموزعة.

تحذير: رموز OAuth مخزنة بنص عادي

نموذج Account يخزن `access_token` و `refresh_token` و `id_token` بنص عادي بدون تشفير. رغم وجود تعليق TODO في الكود، هذا يمثل خطراً أمنياً. يُنصح بتشفيرها باستخدام AES-256-GCM (نفس الطريقة المستخدمة لتشفير `ExchangeCredential`).

الأداء Performance

استعلامات N+1 في مراقبة المراكز

كل مركز يجلب سعره الخاص بشكل منفصل. جزئياً مخفّف عبر `Promise.allSettled` لكن يبقى عدداً كبيراً من استدعاءات API. الحل: تجميع الرموز الفريدة وجلب الأسعار دفعة واحدة (مُنقذ جزئياً في `getMonitorStatus`).

عالي

BullMQ معطل - جميع الأوامر تنفذ متزامنة

بدون قائمة انتظار، لا يوجد backpressure أو إعادة محاولة. إذا تعطل Redis، تستمر الأوامر بالتنفيذ المباشر. هذا مقبول حالياً لكن سيصبح مشكلة مع زيادة المستخدمين.

متوسط

ذاكرة التخزين المؤقت للبرصات غير محدودة

متوسط

exchangeCache هي Map عادية بدون LRU أو TTL. مع مستخدمين كثر وبيانات اعتماد مختلفة، يمكن أن تنمو بلا حدود.

الأمان Security

نقاط الضعف

- OAuth tokens بدون تشفير
- Scanner cron يجلب قوائم مراقبة جميع المستخدمين
- Fail-open عند تعطل Redis في التنسيق
- سجل التدقيق ينمو بلا حدود بدون سياسة احتفاظ

نقاط القوة

- عزل مستوى الصفوف (RLS) في PostgreSQL
- تشفير AES-256-GCM لبيانات الاعتماد
- UserIsolationInterceptor مسجل عالمياً
- Rate limiting ثلاثي المستويات (3/10s, 20s, 100/min)
- WebAuthn (Passkeys) للمصادقة

7 الثغرات الحرجة والتوصيات

تحليل الثغرات حسب الخطورة Bugs by Severity

#1: إغلاق الصفقات عند 4 ساعات بدلاً من 48 ساعة

المسار: الكود القديم على Railway (pre-V176) يغلق صفقات Agent عند 4 ساعات مع `closeReason="Manual"`.

الجذر: كود JS مُجمّع قديم لا يزال يعمل على Railway رغم تحديث الكود المصدري محلياً. الدليل هو أن `closeReason="Manual"` كان القيمة الافتراضية قبل V176.

التأثير: خسارة أرباح الصفقات الرابحة، إحصائيات خاطئة، وثقة منخفضة في النظام.

الحل: (1) إعادة بناء ونشر كامل على (2) Railway. التحقق من أن ملفات JS المُجمّعة محدّثة. (3) إضافة فحص صحة الإصدار في endpoint التشخيصي.

حرج

#2: ExchangeSync يغلق بدون closeReason

المسار: `ExchangeSyncService._closePositionInDB()` يستدعي `closePositionWithRetry()` بدون `closeReason`.

التأثير: المراكز المغلقة عبر مصالحة البورصة تُسجّل بـ `AUTO_CLOSE` أو `MANUAL` بدلاً من `EXCHANGE_SYNC`، مما يصعّب التتبع والتشخيص.

الحل: تمرير `closeReason: 'EXCHANGE_SYNC'` في الاستدعاء.

حرج

#3: تناقض حساب قيمة المحفظة

المسار: `RiskCalculator` يحسب القيمة كـ `paperBalance + unrealizedPnL`، بينما `RiskManager` يحسبها كـ `paperBalance + notional value` الكامل.

التأثير: قرار فتح مركز قد يُرفض من طبقة ويُقبل من أخرى. مع رافعة مالية 10x، الفرق قد يكون 10 أضعاف.

الحل: إنشاء `PortfolioValuationService` موحد تستخدمه جميع طبقات المخاطر.

حرج

#4: paperBalance - V204 قد يكون 0

المسار: `RiskManager._estimatePortfolioValue()` يرجع `paperBalance.toNumber()` بدون قيمة افتراضية.

التأثير: إذا لم يعيّن المستخدم رصيداً ورقياً، القيمة = 0، مما يمنع جميع التداول.

الحل: إضافة `10000 ||` كقيمة افتراضية صريحة قبل try-catch.

عالي

#5: غياب معالجة التنفيذ الجزئي

المسار: النظام يفترض دائماً تنفيذاً كاملاً. `PARTIALLY_FILLED` موجودة لكن غير معالجة.

التأثير: في التداول الحقيقي، قد ينقذ جزء من الطلب فقط، مما يترك مركزاً بحجم غير متوقع بدون إدارة مخاطر مناسبة.

الحل: تنفيذ منطق التنفيذ الجزئي مع إعادة تقديم الجزء المتبقي.

عالي

1 فوري: إصلاح مشكلة النشر على Railway

إعادة بناء ونشر كامل. إضافة فحص صحة الإصدار. التحقق من ملفات JS المُجمّعة. إضافة CI/CD pipeline يمنع نشر كود بدون تجميع جديد.

2 عاجل: توحيد حساب قيمة المحفظة

إنشاء PortfolioValuationService موحد. إصلاح bug `paperBalance = 0`. إضافة اختبارات وحدة لسيناريوهات الحساب المختلفة.

3 مهم: تحسين معالجة الأخطاء والمراقبة

إضافة `closeReason` لكل مسار إغلاق. تنفيذ Partial Fill handling. تشغيل BullMQ لقائمة انتظار الأوامر. إضافة سياسة احتفاظ بسجل التدقيق.

4 تحسين: أداء وتجربة المستخدم

تجميع استعلامات الأسعار. إضافة LRU cache للبورصات. تقليل عدد API routes في Next.js (+50 حالياً). تشفير OAuth tokens.

8 التقييم النهائي والخريطة الرقمية

التقييم التفصيلي حسب الفئة Detailed Category Ratings



التقييم العام

7.2

10/ - جيد مع تحفظات حرجية

خلاصة التقييم Verdict

نظام رؤى يمثل منصة تداول آلي متقدمة ذات تصميم معماري قوي وإدارة مخاطر متعددة الطبقات. نقاط القوة الأبرز هي الفصل الواضح بين نظامي التداول، والتوثيق المكتبي الممتاز، وآليات الحماية الدفاعية كالقفل المتفائل والمصالحة. لكن الثغرة الحرجية في إغلاق الصفقات عند 4 ساعات بدلاً من 48 ساعة - الناتجة عن فجوة بين الكود المحلي والمنشور - تقوّض ثقة المستخدم بشكل كبير. إصلاح هذه المشكلة يتطلب إعادة نشر فورية ومراجعة عملية النشر بالكامل لمنع تكرارها.

على المدى المتوسط، يجب توحيد حساب قيمة المحفظة عبر جميع طبقات المخاطر، وتنفيذ معالجة التنفيذ الجزئي، وتحسين أداء استعلامات الأسعار. على المدى الطويل، يمكن تطوير نظام Backtesting لاستراتيجيات Agent، وتفعيل BullMQ لإدارة ضغط الأوامر، وبناء لوحة مراقبة تشغيلية شاملة. النظام يمتلك أساساً متيناً يبرر الاستثمار في هذه التحسينات.

